Intelligent Traffic Management System Using YOLOv8

AI-based adaptive traffic signal control with emergency vehicle priority

Prepared by: Kesanakurthi Jahnavi

Department: CSE (AI and ML)

Guide: Self

Date: October 2026

1. Abstract

Traditional traffic signals run on fixed timers and manual operation, which causes congestion and delays emergency vehicles. This project builds an AI-based Intelligent Traffic Management System (ITMS) that uses YOLOv8 and OpenCV to detect vehicles and ambulances in video of each lane. The system counts vehicles per lane to estimate traffic density, then dynamically sets the green light duration. When an ambulance is detected with high confidence, its lane receives an immediate green signal. A Flask web dashboard with secure login shows the live video, lane counts, signal states and analytics, and stores records in an SQLite database.

Keywords: YOLOv8, computer vision, OpenCV, traffic density, adaptive signal control, emergency vehicle priority, Flask, SQLite.

2. Introduction

An Intelligent Traffic Management System uses sensing and automation to manage road traffic better than manual or timer-based methods. This project uses camera video as the sensor. A deep learning object detector identifies vehicles frame by frame, and a control algorithm converts the detections into signal decisions.

The system also gives priority to ambulances, which is a practical need because emergency vehicles lose critical time at congested junctions.

3. Problem Statement

Fixed-timer signals give each lane the same green time regardless of how many vehicles are waiting. They cannot recognize ambulances, collect traffic data, or be monitored from a central place. The result is longer waiting times, wasted green time on empty lanes, and delayed emergency response.

4. Objectives

- Detect and count vehicles in each lane in real time using YOLOv8.
- Detect ambulances using a custom-trained model.
- Adjust green light timing automatically based on lane density.
- Override the normal cycle to clear the path for an ambulance.
- Provide a secure web dashboard for live monitoring and analytics.
- Store traffic records so they can be analyzed and exported.

5. Existing System

- Manual or fixed-timer traffic signals.
- No real-time data collection.
- No emergency vehicle priority; handling is manual.
- Inefficient traffic flow during peak hours.

- No centralized monitoring or analytics.

6. Proposed System

- **Real-time object detection:** YOLOv8 detects and classifies cars, motorcycles, buses and trucks.
- **Traffic logic automation:** a controller sets green time from live density.
- **Ambulance override:** a second, custom-trained YOLOv8 model detects ambulances and forces green for that lane.
- **Analytical dashboard:** a Flask web app shows the live stream, lane counts and charts.
- **Database integration:** SQLite stores users and periodic traffic records.

Comparison

Parameter	Existing system	Proposed system
Signal timing	Fixed timing	Dynamic, density-based
Ambulance handling	Manual	Automated priority
Monitoring	None	Real-time dashboard
Data records	None	Stored in SQLite, CSV export
Technology	Rule-based timer	YOLOv8 computer vision

7. Software and Hardware Requirements

Software

- Operating system: Windows 10/11
- Language: Python 3.11, HTML, CSS, JavaScript
- Libraries: ultralytics (YOLOv8), opencv-python, Flask, Werkzeug, SQLite3 (built in), Chart.js (charts)
- Editor: Visual Studio Code
- Model training: Google Colab (T4 GPU) with a Roboflow dataset

Hardware

- Processor: Intel i3 or better
- RAM: 8 GB recommended (4 GB minimum)
- Storage: 20 GB free
- Input devices: keyboard, mouse
- Camera or recorded traffic video as input
- A GPU is optional; detection runs on CPU but more slowly

8. System Architecture

The system has these layers:

1. **Input layer:** traffic video file (or CCTV feed) read frame by frame with OpenCV.
2. **Detection layer:** YOLOv8 vehicle model (yolov8n.pt) and a custom ambulance model (best.pt).
3. **Logic layer:** the lane counter and the signal controller (traffic_logic.py).
4. **Application layer:** Flask server with login, dashboard, analysis page, video stream, upload and report routes.
5. **Data layer:** SQLite database (database.db) with users and traffic_log tables.
6. **Presentation layer:** HTML dashboard showing the live feed, lane cards and charts.

Data flow: video frame, then YOLOv8 detection, then lane assignment and counting, then signal controller, then dashboard and database.

9. Methodology and Modules

9.1 User Authentication Module

Handles registration and login with role-based access (admin and user). Passwords are stored as hashes using Werkzeug, not as plain text. The first registered account becomes the admin. The analysis page is restricted to admins.

9.2 Object Detection Module

YOLOv8 (yolov8n.pt) processes each frame. Only the vehicle classes car, motorcycle, bus and truck are counted. Detections are drawn as boxes on the frame.

9.3 Traffic Density Calculation Module

The frame is divided into four equal vertical strips, one per lane. Each detected vehicle is assigned to a lane using the horizontal centre of its bounding box. The number of vehicles per lane is the density value.

9.4 Traffic Logic Module

A controller selects the next green lane when the current green time ends. It picks the busiest lane other than the current one, so one lane cannot hold green forever. Green time is calculated as:

`green time = 10 s + 2 s x vehicle count`, limited to a maximum of 40 s.

9.5 Ambulance Priority Module

A second YOLOv8 model, trained on an ambulance dataset, runs on every third frame to save processing time. If an ambulance is detected with confidence of 0.7 or higher, the controller immediately makes that lane green and holds it for 15 seconds. The dashboard shows an alert banner.

9.6 Video Input and Upload Module

The system plays a default traffic video. A user can upload another video (mp4, avi or mov) from the dashboard so that it can be analysed. Switching the live feed to the uploaded video is a known issue (see Section 14).

9.7 Dashboard and Analytics Module

The Flask dashboard streams the processed video using MJPEG and refreshes lane counts, green lane, countdown and ambulance alert every second through a JSON API.

9.8 Traffic Analysis and Report Module

A record of lane counts, green lane and ambulance status is saved every 5 seconds. The analysis page plots lane counts over time with Chart.js and shows a summary. The data can be downloaded as a CSV report.

10. Ambulance Model Training

- **Dataset:** ambulance images from Roboflow Universe, exported in YOLOv8 format.
- **Platform:** Google Colab with a T4 GPU.
- **Base model:** yolov8n.pt, fine-tuned for 30 epochs at image size 640.
- **Output:** best.pt, the best checkpoint, used by the application.

11. Database Design

Table	Columns
users	id, username (unique), password (hashed), role
traffic_log	id, time stamp, counts (JSON list of four lane counts), green lane, ambulance flag

12. Application Routes

Route	Purpose
/register, /login, /logout	Account management
/	Live dashboard
/video-feed	Processed MJPEG stream
/api/stats	Current counts and signal state
/api/history	Last 60 records for the chart
/analysis	Charts and summary (admin only)
/upload	Upload a new video
/report.csv	Download the CSV report

13. Implementation Steps

1. Set up Python 3.11, VS Code and the required libraries.
2. Test YOLOv8 vehicle detection on a traffic video.
3. Add lane-wise vehicle counting.
4. Build the signal controller with density-based timing.
5. Train the ambulance model on Colab and add the override.
6. Build the Flask app with a live video stream and dashboard.
7. Add login, SQLite logging, the analysis page, video upload and CSV report.

14. Results and Testing

The system was tested on recorded traffic videos. The status of each feature is given below.

Working

- **Vehicle detection and counting:** vehicles are detected and assigned to lanes, and the lane counts update live on the video and the dashboard.
- **Signal control:** the green lane changes automatically, with the countdown shown on the dashboard.
- **Web application:** registration, login, role-based access, the live dashboard, the analysis chart and the CSV report.

Partially tested or known issues

- **Ambulance override:** the custom model and the override logic are integrated, but they need more testing with real ambulance videos before the detection performance can be reported.
- **Video upload:** the upload feature is implemented, but the live feed does not yet switch reliably to the newly uploaded video. This is being fixed.

Quantitative accuracy measurements on a larger labelled test set have not been carried out and are left as future work.

15. Advantages

- Green time adapts to real traffic density, reducing waiting.
- Automatic ambulance priority without manual intervention.
- Real-time monitoring through a web dashboard.
- Stored data supports analysis and reporting.
- Uses low-cost hardware and open-source tools.

16. Limitations

- Lanes are fixed vertical strips, so the camera angle must suit this layout.
- Accuracy depends on video quality, lighting and weather.

- The ambulance model depends on the size and variety of its training dataset.
- The project controls a simulated signal on screen and is not connected to physical signal hardware.
- Detection is slower on CPU-only computers.

17. Future Scope

- Connect to real signal controllers through a microcontroller or relay.
- Support multiple cameras and junctions with a central control server.
- Add vehicle tracking to avoid double counting and measure speed.
- Add siren sound detection to improve ambulance detection.
- Add number plate recognition and violation detection.
- Predict traffic patterns using historical data.
- Deploy on edge devices with a GPU accelerator.

18. Conclusion

The project shows that computer vision can replace fixed-timer signals with an adaptive system. YOLOv8 provides real-time vehicle and ambulance detection, the controller adjusts green time by lane density, and the Flask dashboard with SQLite storage gives monitoring and analytics. The system gives emergency vehicles priority automatically, and it can be extended to real-world deployment with more hardware integration and testing.

19. References

1 Ultralytics YOLOv8 documentation, docs.ultralytics.com

2 OpenCV documentation, docs.opencv.org

3 Flask documentation, flask.palletsprojects.com

4 Roboflow Universe, universe.roboflow.com (ambulance dataset)

5 SQLite documentation, sqlite.org

6 Chart.js documentation, chartjs.org